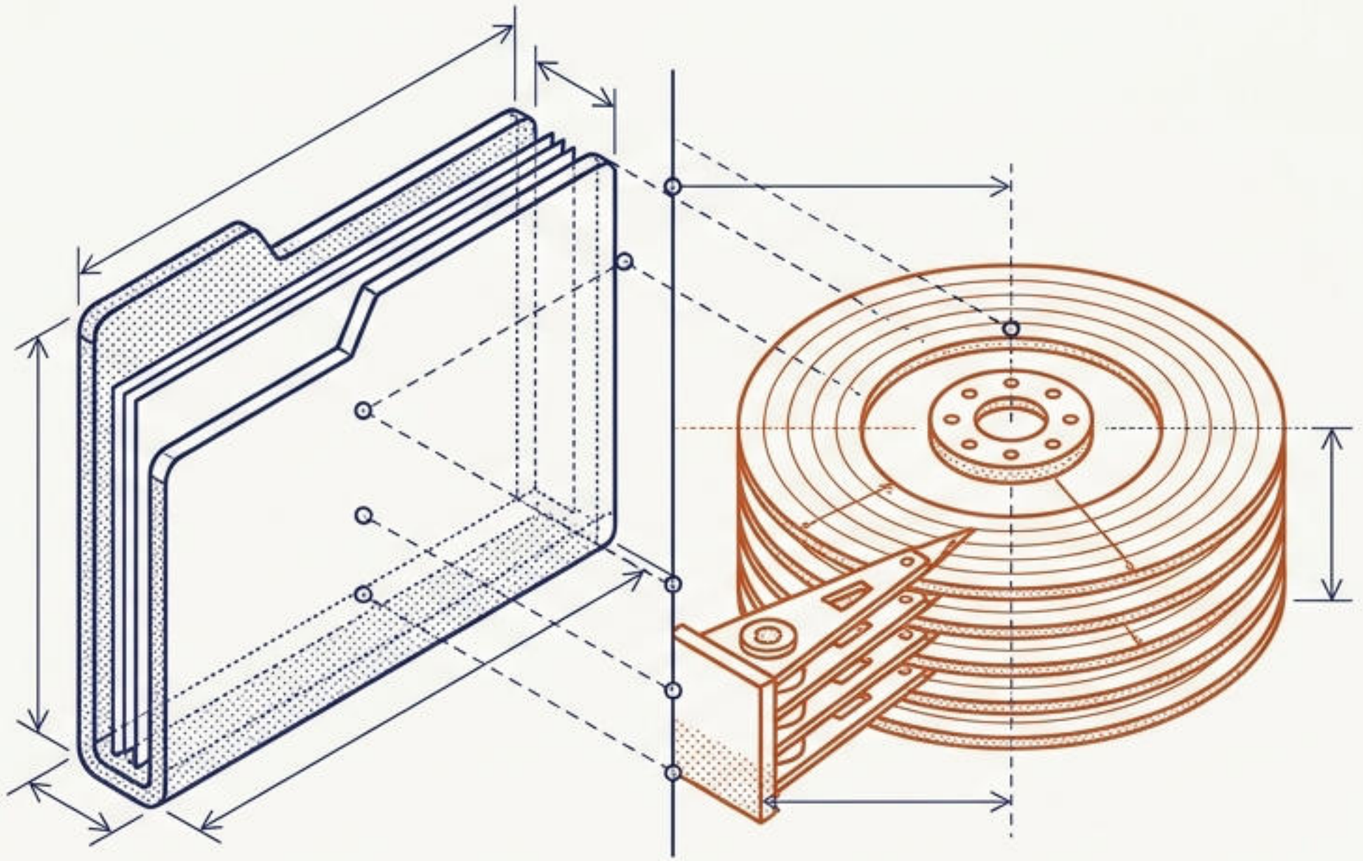


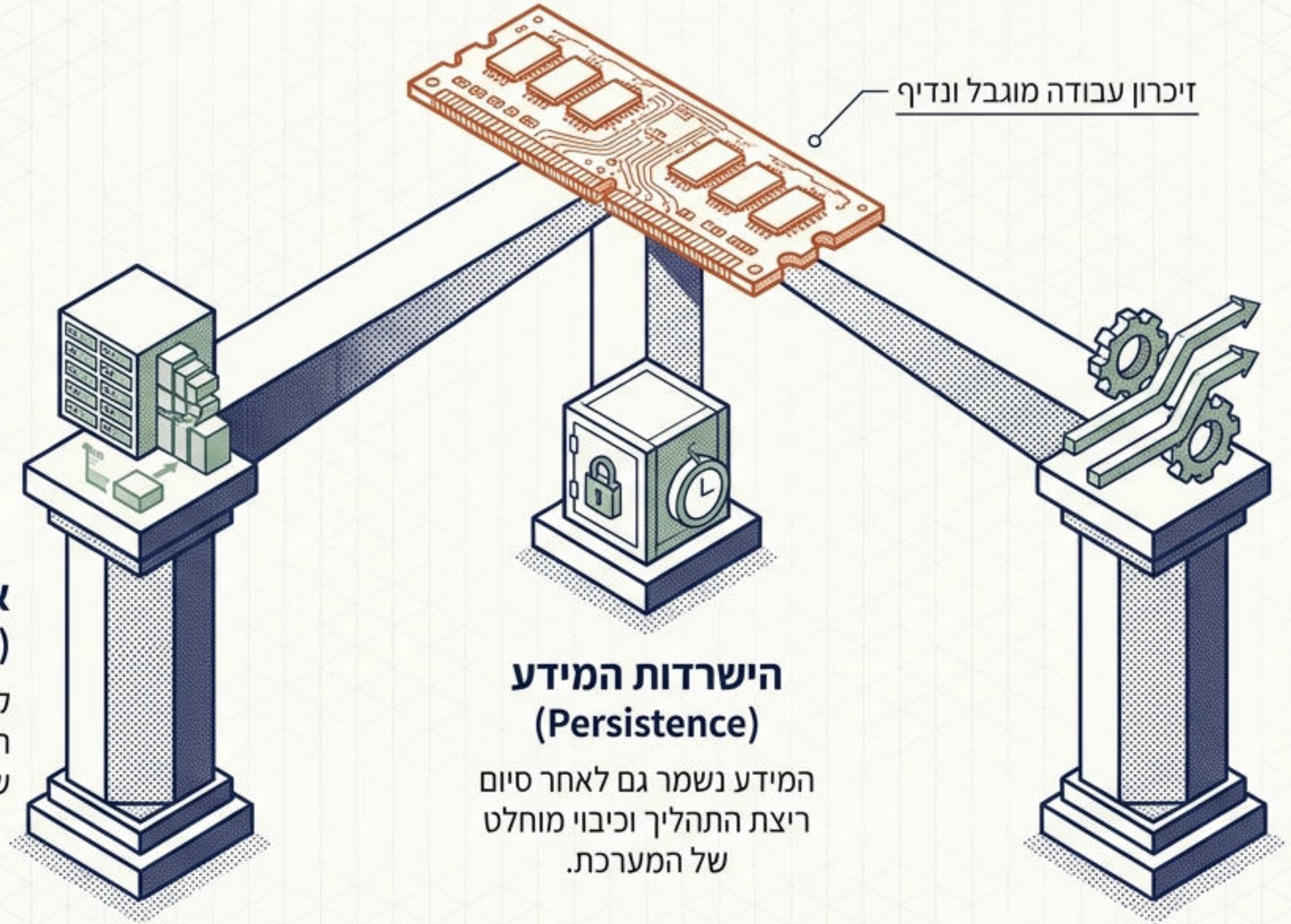
מערכות קבצים (File Systems)

מבט רנטגן מקיף: מהפשטה לוגית לארכיטקטורה פיזית עמוקה (כרך א', פרק 4).

מדריך מאסטר לניהול דיסקים, מדריכים, מנגנוני הקצאה, ואופטימיזציה של ביצועים.

https://operating-system-5l3.pages.dev/

הפער הפיזי: מדוע נדרשת מערכת קבצים?



אחסון עצום (Massive Capacity)

קיבולת אדירה שחוצה את הגבולות הפיזיים המצומצמים של הזיכרון הראשי.

הישרדות המידע (Persistence)

המידע נשמר גם לאחר סיום ריצת התהליך וכיבוי מוחלט של המערכת.

גישה מקבילית (Concurrency)

מנגנון בטוח המאפשר למספר תהליכים לגשת למידע בו-זמנית מבלי לשבש נתונים.

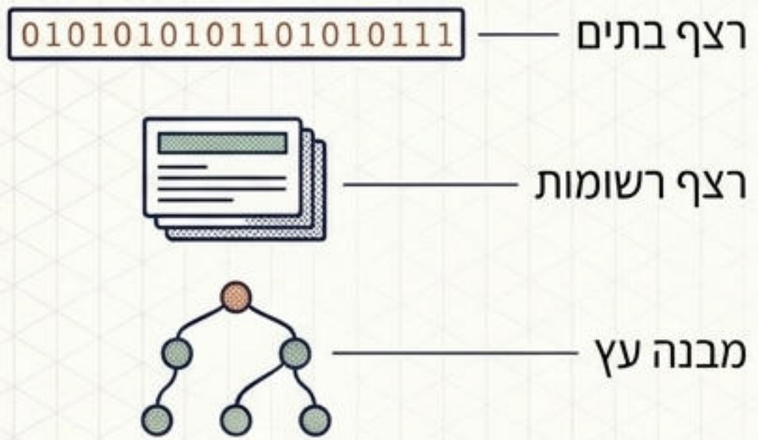
האנטומיה של קובץ: מבנה ומהות

עקרון הברזל: Mechanism vs. Policy



מערכת ההפעלה מספקת רק את המנגנון (Mechanism), בעוד שארגון התוכן (Policy) באחריות התהליך.

מבנה פנימי



טיפוסי קבצים



התקן בלוקים (Block)

הפשטה להתקני גישה אקראית כמו דיסק.



(Character Special)

הפשטה לחומרה טורית כגון מקלדת.



מדריכים (Directories)

קובצי מערכת לשמירת הסדר ההיררכי.



רגילים (Regular)

נתוני משתמש (טקסט או בינארי).

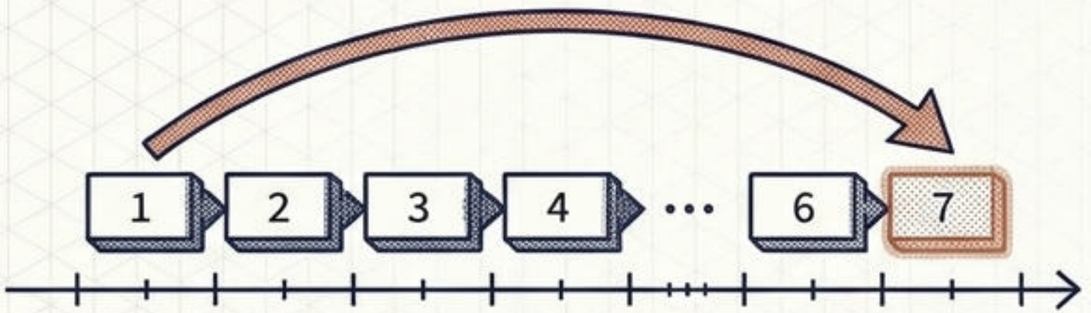
גישה, מטא-נתונים ומנגנוני הגנה

שיטות גישה



סדרתית (Sequential)

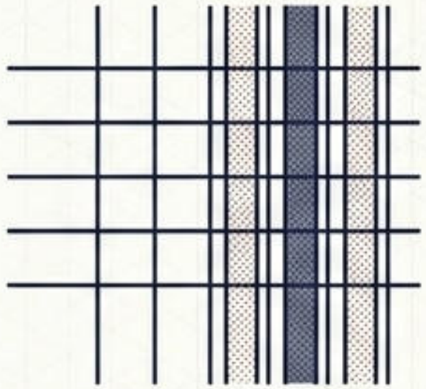
קריאה בית אחר בית מתחילת הקובץ. חובה לאפס כדי לחזור אחורה.



אקראית (Random)

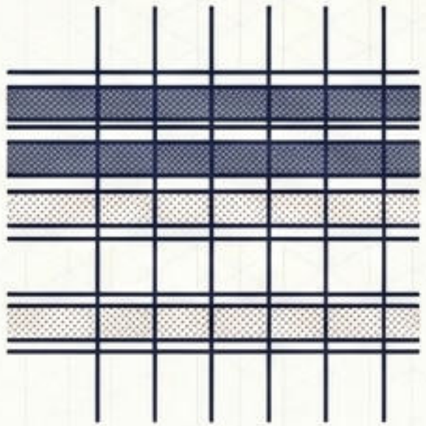
דילוג (Seek) ישירות לכל מיקום. קריטי למסדי נתונים ולדיסקים.

פיצול מטריצת הגישה (R, W, X)



רשימות גישה (ACL)

שמירה לפי עמודות.
לכל קובץ רשימת משתמשים מורשים.





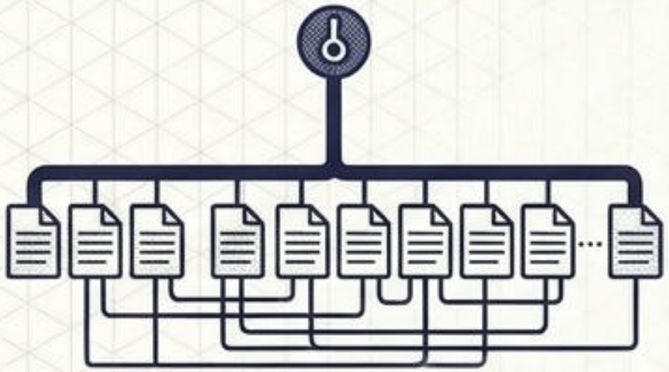
רשימות יכולות (C-Lists)

שמירה לפי שורות.
לכל תהליך 'מחזיק מפתחות' לקבצים מורשים.

מטא-נתונים (Metadata): הרשאות, זמן יצירה, ומונה הצבעות נשמרים בנפרד מהתוכן (ב-node-).

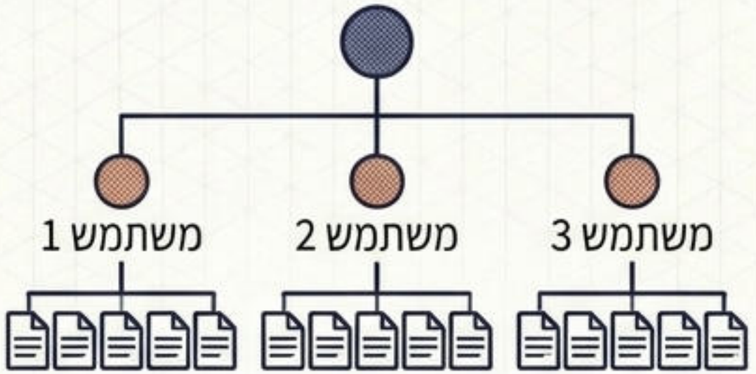
https://operating-system-5i3.pages.dev/

אבולוציית המדריכים וניווט



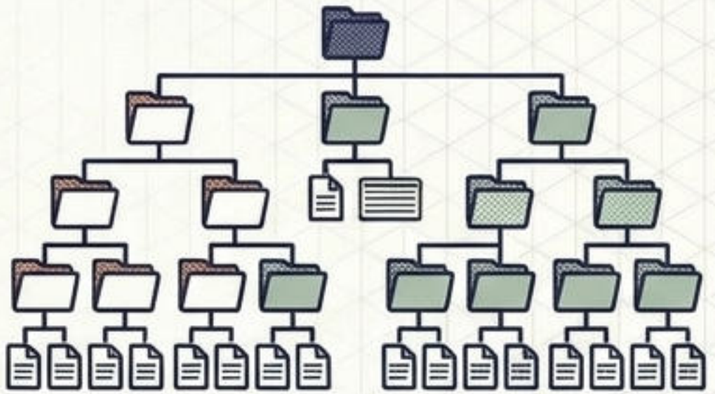
1. היררכיה חד-מפלסית

שורש משותף. חיסרון: התנגשות שמות (Name Collision).



2. היררכיה דו-מפלסית

מדריך לכל משתמש. פותר התנגשויות בין משתמשים.



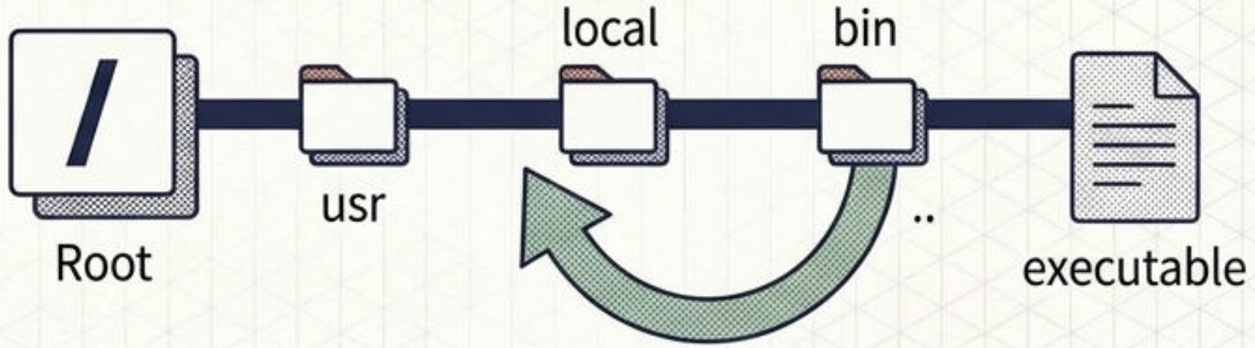
3. מערכת היררכית (עץ)

המודל המודרני. עומק בלתי מוגבל המעניק שליטה וארגון מושלם.

נתיבים וניווט

נתיב מוחלט (Absolute): יוצא תמיד משורש המערכת (/). עמיד לשינויי מיקום.

נתיב יחסי (Relative): מתחיל מהמדריך הנוכחי. מבוסס על '.' (נוכחי) ו- '..' (ספריית האב).

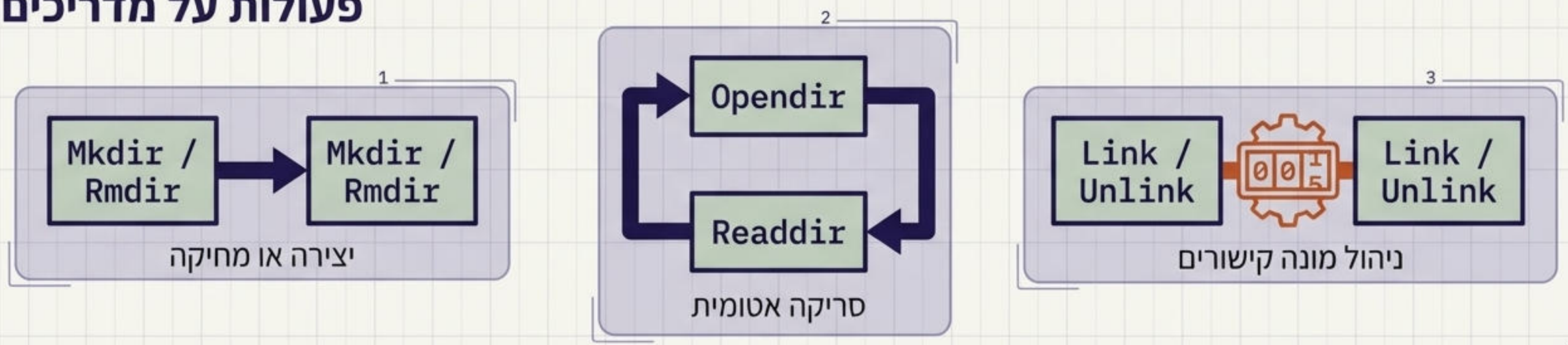


קריאות מערכת (System Calls): פאנל השליטה

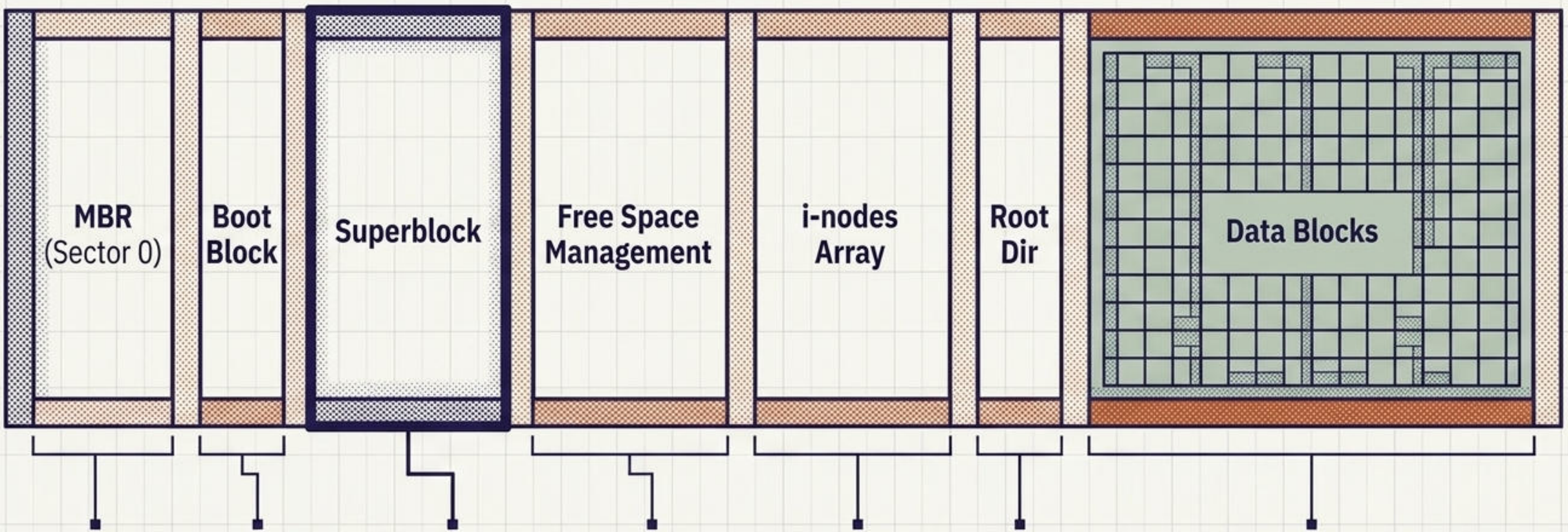
פעולות על קבצים



פעולות על מדריכים

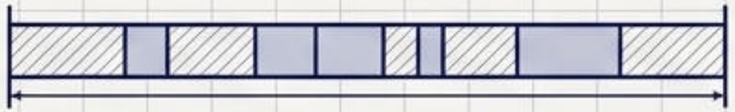


הגישור הפיזי: פריסת מערכת הקבצים על הדיסק



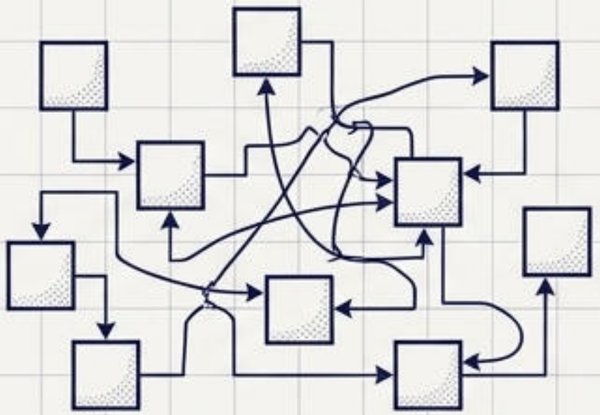
דילמת ההקצאה (Allocation): היכן שומרים את הבלוקים?

1. הקצאה רציפה (Contiguous)



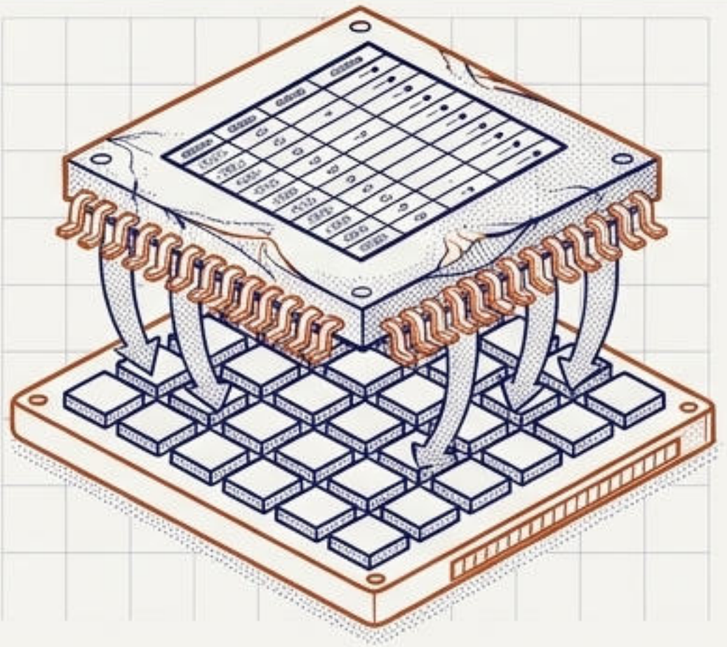
- הקובץ נשמר כרצף אחד של בלוקים עוקבים.
- **יתרון:** ביצועים פנומנליים (מינימום Seek Time).
- **חיסרון:** קיטוע חיצוני חמור (חורים) וקושי בהגדלת קבצים.

2. רשימה מקושרת (Linked List)



- בלוקים מפוזרים חופשי, כל אחד מצביע לבא.
- **יתרון:** 0% קיטוע חיצוני.
- **חיסרון:** גישה אקראית אטית מאוד ואובדן שטח למצביעים.

3. טבלת מפות (FAT)

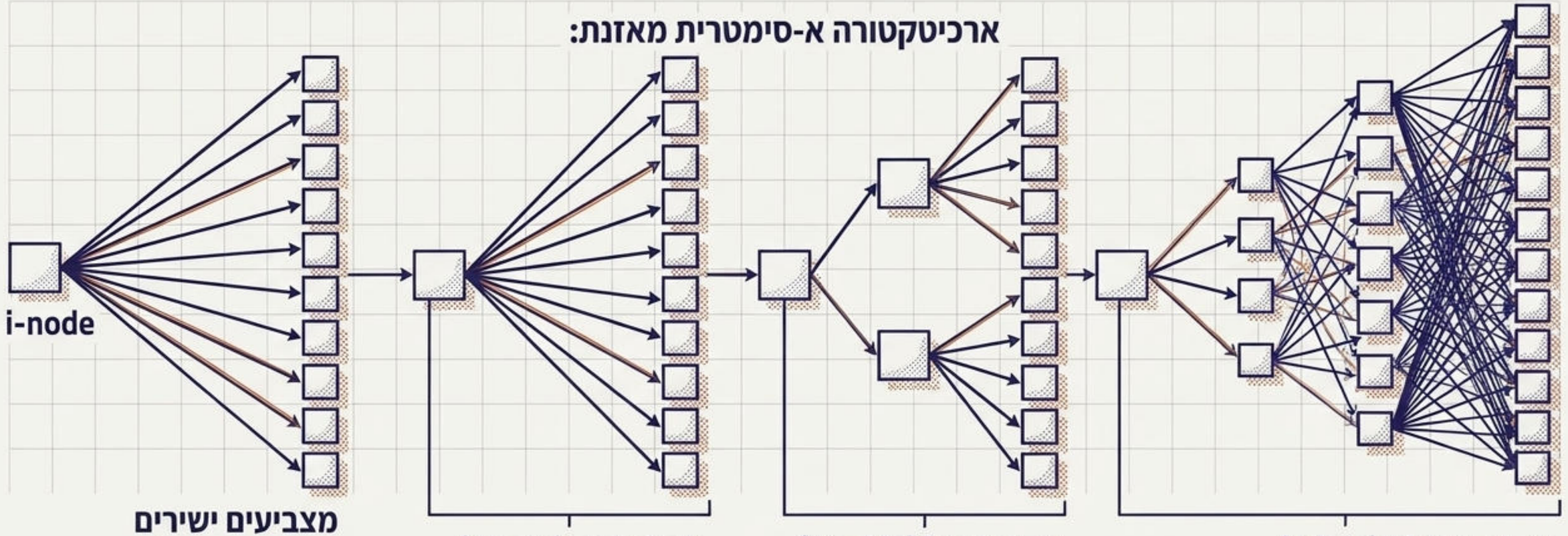


- ריכוז שרשרת המצביעים בטבלה ב-RAM לגישה אקראית מהירה.
- **החיסרון הקריטי** (קריסה): צריכת זיכרון בלתי אפשרית לדיסקים מודרניים גדולים.

יצירת המופת: ה-UNIX i-node כפתרון אולטימטיבי

הקונספט המהפכני: טעינת המטא-נתונים ל-RAM רק עבור קבצים הפתוחים כרגע, במקום טבלה שלמה לכל הדיסק.

ארכיטקטורה א-סימטרית מאזנת:



מצביעים ישירים
10-12 הצביעות למידע. מרדע. מכסה 90% מהקבצים הקטנים במהירות מקסימלית.

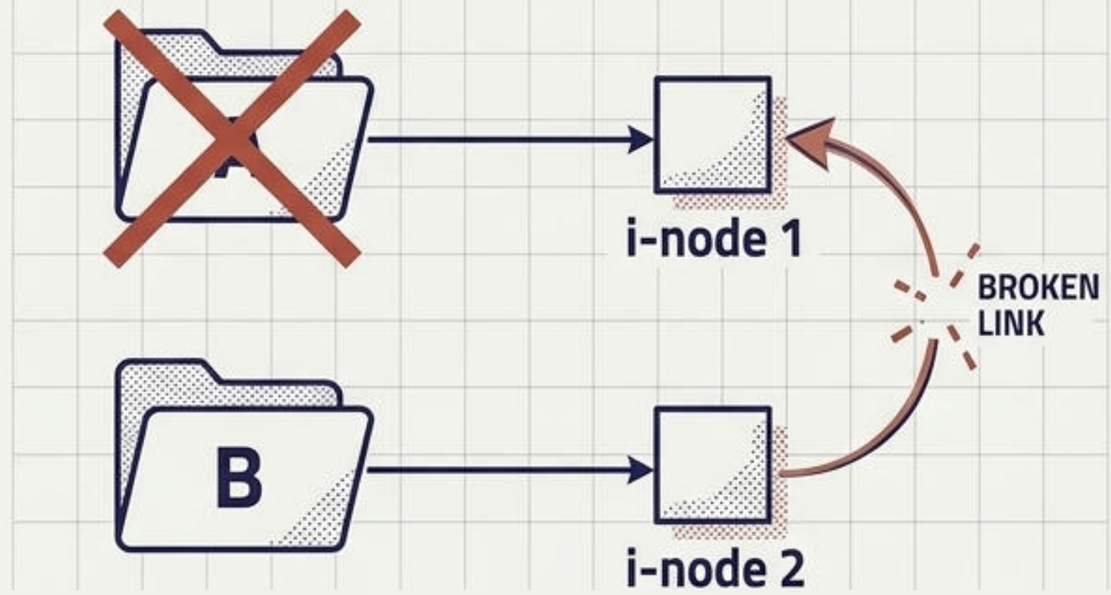
עקיף יחיד (Single)
מצביע לבלוק שכולו טבלת מצביעים ישירים.

עקיף כפול (Double)
מצביע לבלוק של מצביעים לטבלאות (לקבצי GB).

עקיף משולש (Triple)
רמה נוספת המאפשרת צמיחה עצומה (טרה-בייטים) מאותו מבנה קומפקטי יחיד.

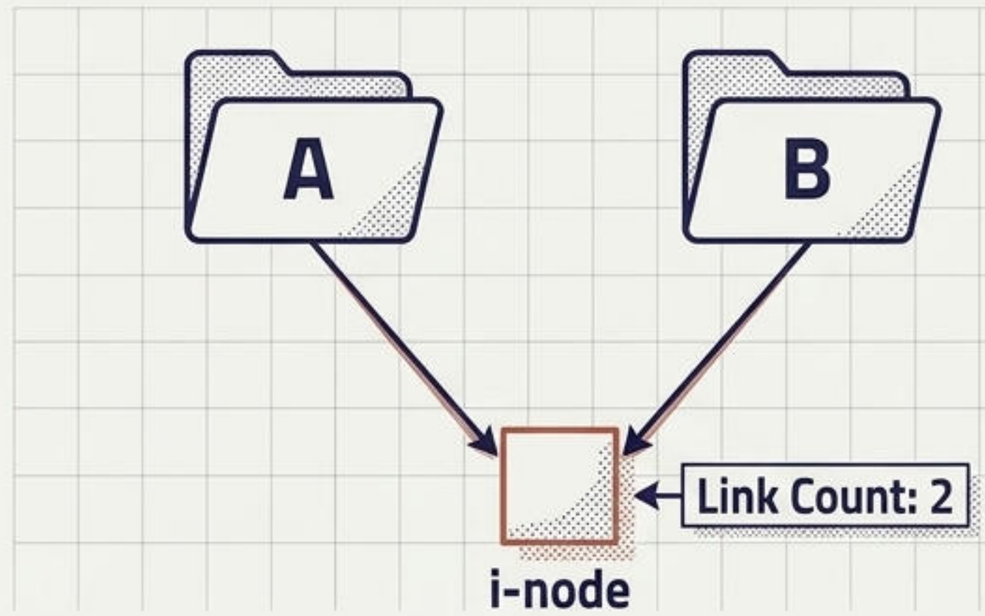
שיתוף קבצים וקישורים: אשליות זהות מול מצביעים

קישור סמלי (Soft/Symbolic Link)



- קובץ חדש לחלוטין (עם i-node משלו), שתוכנו הוא מחרוזת נתיב למקור.
- **יתרון:** ניתן לקשר בין מחיצות ואף שרתים מרוחקים.
- **סכנה:** אם המקור נמחק פיזית, הקישור נשאר יתום ונשבר (Dangling Pointer).

קישור קשיח (Hard Link)



- שתי רשומות מדריך המצביעות לאותו i-node פיזי בדיוק.
- ה-Link Count עולה. הקובץ נמחק פיזית רק כשהמונה שווה 0.
- **מגבלה:** לא חוצה מחיצות דיסק ואסור על ספריות.

ניהול שטח וקיטוע: מאזן הכוחות הפיזי



בלוק קטן (למשל 512B)

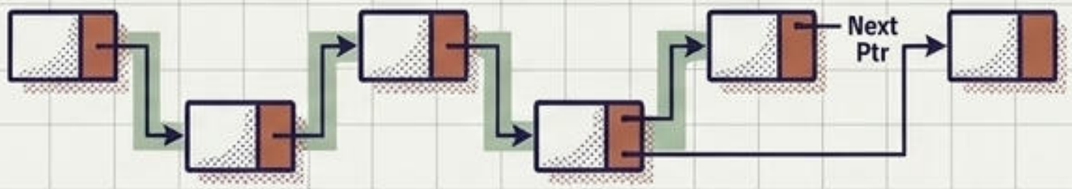
מונע בזבוז שטח, אך פוגע בביצועים בשל ריבוי גישות לדיסק.

בלוק גדול (למשל 32KB)

משפר ביצועי קריאה, אך גורם לקיטוע פנימי (Internal Fragmentation) – בזבוז שטח לקבצים קטנים.

מעקב אחר מקום פנוי

רשימה מקושרת (Linked List)



חוסכת RAM, אך אטית מאוד לחיפוש רצפים בשל קריאות דיסק תכופות.

מפת סיביות (Bitmap)

1	1	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	1	1	0	0	0	0	1	1	0	0	0	0
0	1	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	0	1	1	1	0	0	0	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0
0	0	1	1	1	1	1	0	0	0	0	1	1	0	1	1	1	1	1	0	0	0	0	0	1	0	0	0	0	0
1	0	0	1	1	0	1	0	0	0	1	1	1	1	0	0	1	1	0	0	0	1	0	0	0	1	0	0	1	1

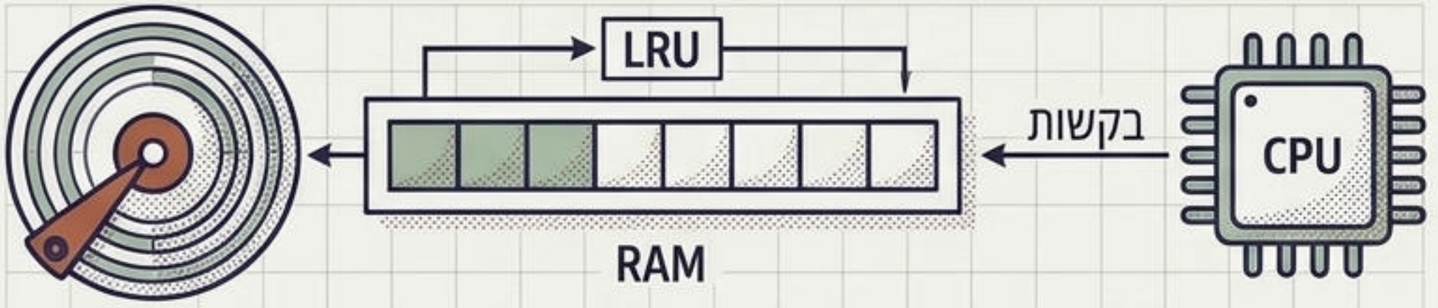
סריקה סופר-מהירה במעבד לאיתור רצפים, אך גוזלת RAM בדיסקי ענק.

הפיתרון המערכתי: קבוצות בלוקים (Block Groups)

חלוקת הדיסק לאזורים גיאוגרפיים עם מפת סיביות עצמאית לכל אזור.

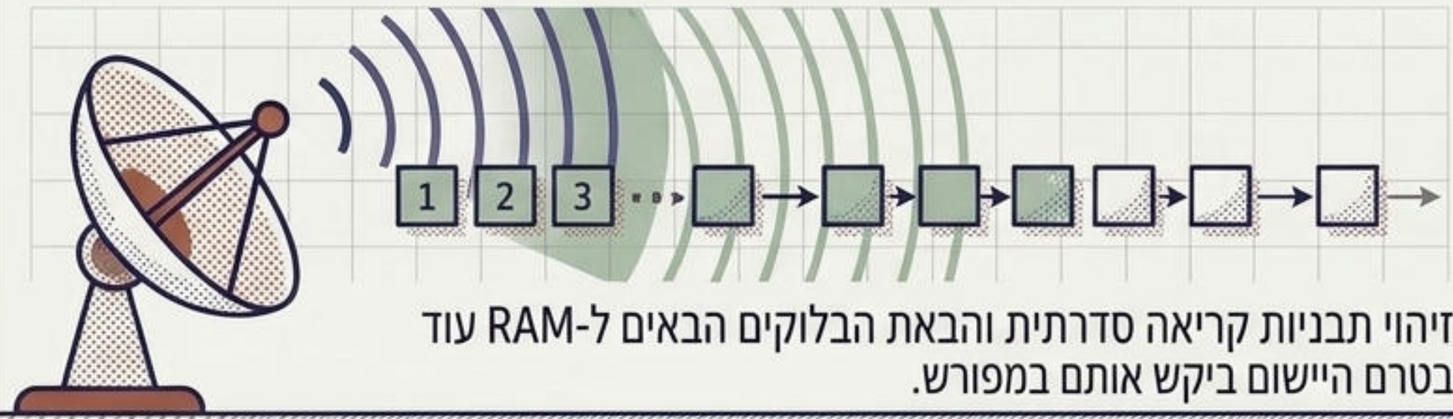
אופטימיזציה של ביצועים: עקיפת איטיות הדיסק

1. מטמון בלוקים (Buffer Cache)



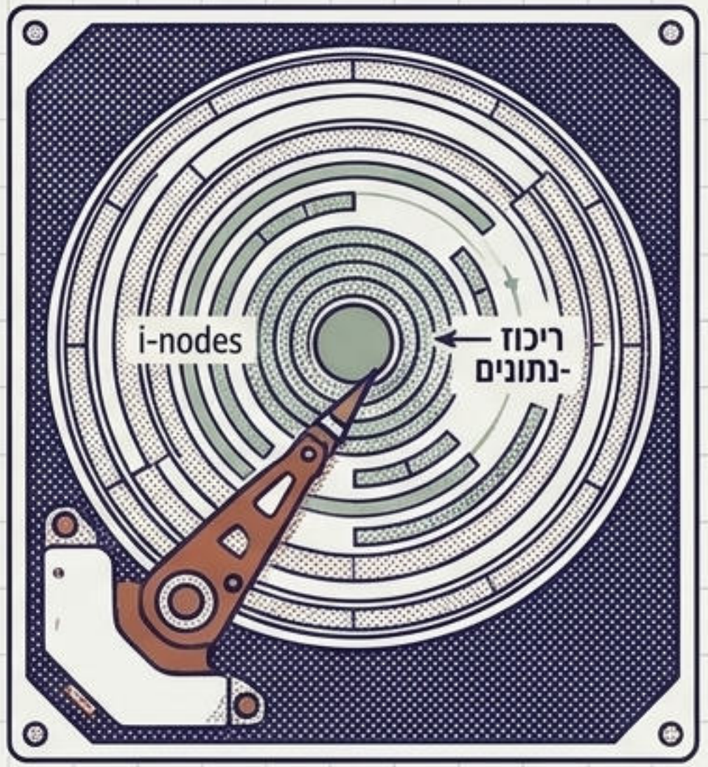
שמירת בלוקים מבוקשים ב-RAM (ניהול LRU). חובה לבצע סנכרון (Sync) תקופתי של המטא-נתונים לדיסק למניעת אובדן.

2. קריאה מראש (Prefetch / Read Ahead)



זיהוי תבניות קריאה סדרתית והבאת הבלוקים הבאים ל-RAM עוד בטרם היישום ביקש אותם במפורש.

3. צמצום תנועות זרוע (Arm Movement)



ריכוז ה-i-nodesi באמצע הדיסק, או הלוקה לקבוצות צילינדרים כך שכל קובץ ייכתב קרוב ככל האפשר למטא-נתונים שלו (הקטנת Seek Time).

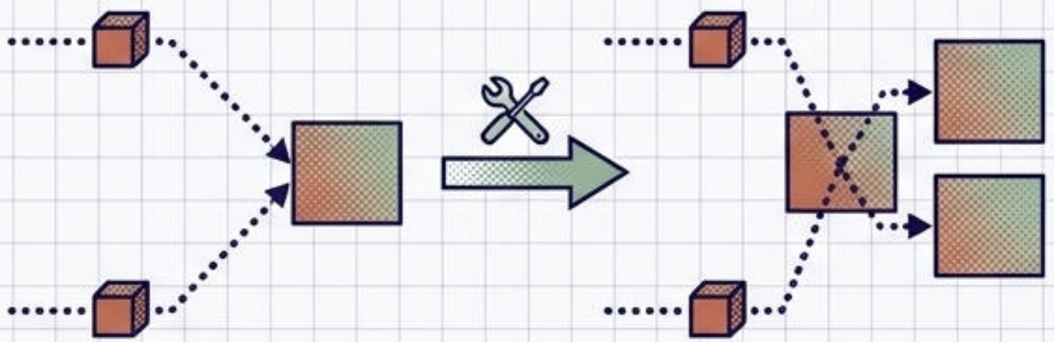
שרידות, גיבויים והתאוששות מאסון (fsck)

אסטרטגיות גיבוי (Dumps)



בדיקת עקביות מערכת (fsck)

תיקון עקביות בלוקים



איתור בלוק המופיע בשני קבצים שונים, והעתקתו (שכפולו) לבלוק חדש כדי להפריד ביניהם ולמנוע דריסה.

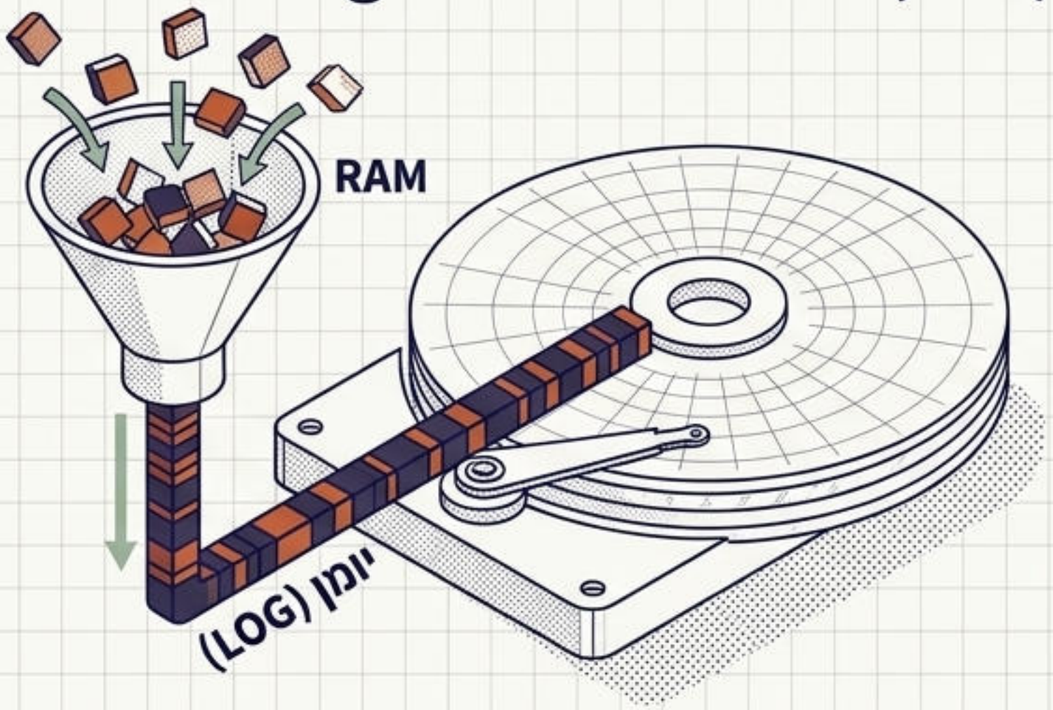
תיקון עקביות ספריות



השוואת כמות הקישורים הפיזיים למונה הקישורים (Link Count) ב-i-node. עדכון המונה כלפי מעלה כדי למנוע מחיקת מידע בטעות.

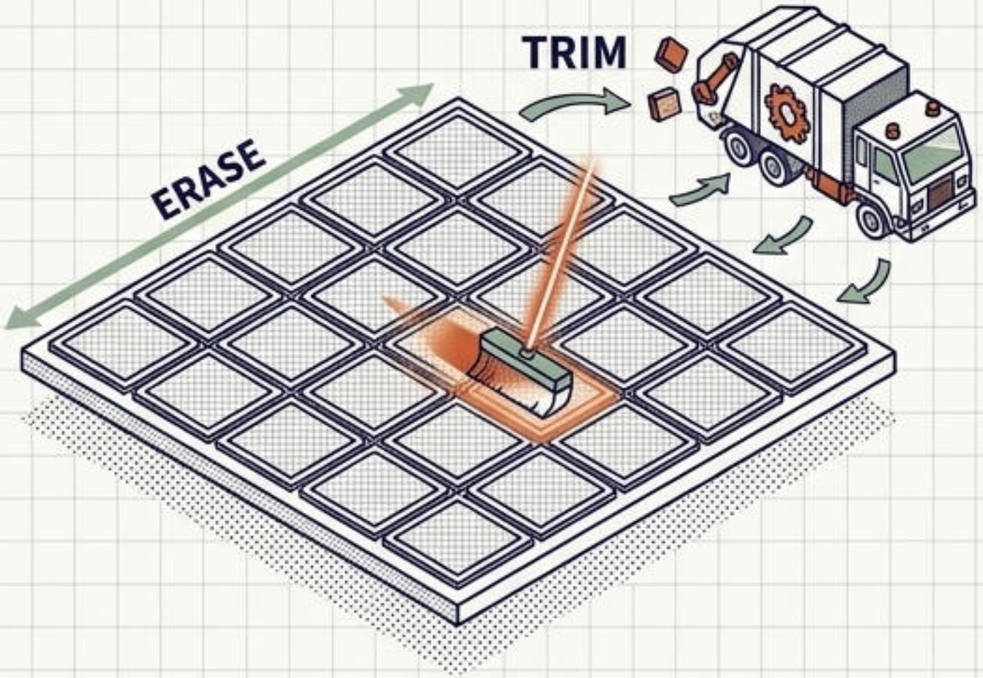
חזית הטכנולוגיה: LFS וכוננ מצב מוצק (SSD)

Log-Structured FS (LFS)



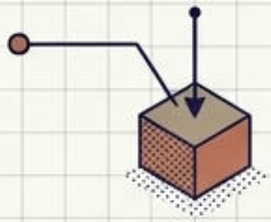
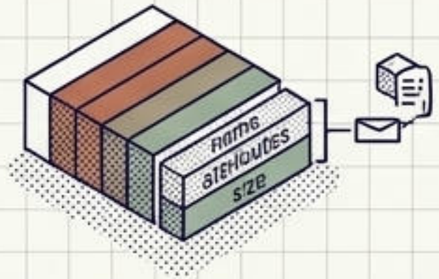
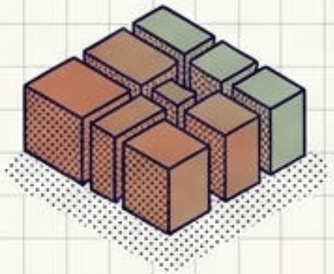
- המערכת אוגרת כתיבות ב-RAM וכותבת אותן במכה אחת סדרתית לסוף הדיסק כ'יומן' ארוך.
- מבטל לחלוטין זמן זרוע. דורש אוסף אשפה (GC) לניקוי יומנים ישנים.

מערכות פלאש (F2FS / SSD)



- אין Seek Time, אך קיימת מגבלת חיי כתיבה וחובה למחוק בלוק לפני כתיבה אליו.
- המערכת לעולם אינה דורסת במקום, אלא כותבת לבלוק חדש פנוי.
- פקודת TRIM: המערכת מודיעה מראש על מחיקות לוגיות כדי שהבקר יבצע 'פינוי אשפה' מראש.

סינתזה היסטורית: הארכיטקטורה מכתיבה יכולות

	UNIX V7 (עידן ה-i)	MS-DOS (משפחת FAT)
מבנה רשומת ספרייה	 16 בתים בלבד (שם (שם ומזהה -i-node). המטא-נתונים שמורים מרכזית.	32 בתים (שם, מאפיינים, גודל, תאריכים). כל המטא-נתונים בספרייה. 
ניהול ואתגרים	 ארכיטקטורה **א-סימטרית** התומכת ב-0 בזבוז לקבצים קטנים ובמקביל בגדילה אינסופית כמעט.	 עם גדילת הדיסקים אולצו להגדיל את **הבלוק** (עד 32KB), מה שיצר **קיטוע פנימי** הרסני.

סיכום מאסטר

מערכת הקבצים אינה רק אוסף תיקיות; היא **הגשר ההנדסי** המתרגם את כוונת המשתמש לחוקי הפיזיקה של אמצעי האחסון.